

Lab 3 – Architectural Justification

Remote Team Time-Tracking & Project Approver (RTTPA)

Shreya Bonal | PES2UG24CS485 | Class 5H

Architecture Selection

We chose the Layered Architecture for the Remote Team Time-Tracking & Project Approver system. The system is organised into a Presentation Layer (User Interface Component), a Business Logic Layer (Authentication & User Management, Project & Task Manager, Time Tracking, Approval Service, Reporting & Notification) and a Data Layer (Database Component).

Reason 1 – The workflow is naturally tiered

Every feature in our backlog follows the same path: a remote team member acts on the screen, a business rule is applied, and the result is stored. Logging hours, submitting a timesheet, approving or rejecting it and generating a report all reuse the same request path, so separating presentation, logic and storage removes duplicated code and lets one epic be built or changed without touching the others.

Reason 2 – Single source of truth for timesheet data

Time entries, approvals and reports must always agree with each other; a report that disagrees with an approved timesheet is a defect. A layered design keeps one Data Layer and one shared IDataAccess interface, so every business component reads and writes the same records. A microservices split would give each service its own store and force us to solve eventual-consistency problems that a student-scale project does not need.

Security Advantage

No component in the Presentation Layer talks to the database directly. The UI can only call the published business interfaces (IUserAuth, IProjectTask, ITimeLog, IApproval, IReporting), and only the Business Logic Layer holds the socket to IDataAccess. This gives a single choke point where authentication and role checks are enforced — the Approval Service can verify that the caller is actually a project manager before any timesheet is marked approved, so a team member cannot approve their own hours even if the client request is tampered with.

Performance Benefit

Because all database access is funnelled through one interface, connection pooling and caching can be applied in a single place. Frequently read but rarely changed data — user profiles, project and task lists — is cached in the Business Logic Layer, so the high-frequency operations (start timer, stop timer, view logged hours) are served without repeated database round trips. Reporting queries are also isolated in their own component, so a heavy report does not block the time-tracking path used by the rest of the remote team.

Components and Interfaces

Components (7): User Interface, Authentication & User Management, Project & Task Manager, Time Tracking, Approval Service, Reporting & Notification, Database.

Interfaces (6): IUserAuth, IProjectTask, ITimeLog, IApproval, IReporting (provided to the UI) and IDataAccess (provided by the Database Component to all business components). Four «use» dependencies link the business components to each other.